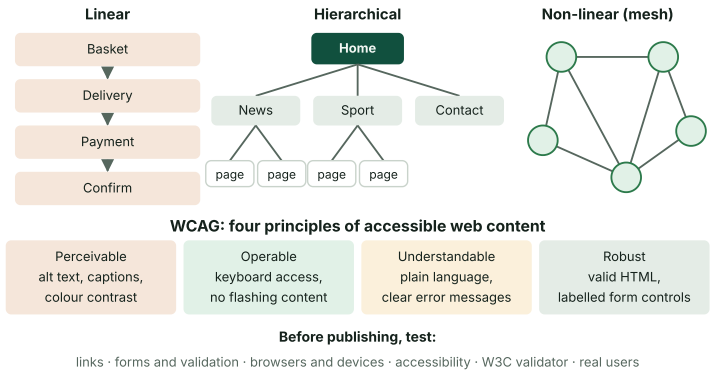
# Website design, client-side and server-side processing, testing and publishing

FD19 FACT FILE

## Specification: students should be able to

- Describe how a website is planned, including purpose, target audience, site maps and storyboards
- Describe linear, hierarchical and non-linear navigation structures and principles of usability
- Explain how websites can be made accessible, with reference to the Web Content Accessibility Guidelines (WCAG)
- Distinguish between static web pages and web applications, and between client-side and server-side processing
- Explain why form input should be validated both on the client and on the server
- Describe how a website is tested, published and maintained

**Key words:** target audience, site map, storyboard, wireframe, navigation structure, hierarchical structure, linear structure, usability, WCAG, responsive design, static web page, web application, client-side processing, server-side processing, validation, broken link, domain name, web hosting




Linear, hierarchical and non-linear navigation structures, and the four WCAG principles: perceivable, operable, understandable and robust

## Planning a website

Building a good website starts long before any HTML is written. The developer first agrees the purpose of the site, such as informing parents about a school, selling tickets for a theatre or taking bookings for a leisure centre, and identifies the target audience: their age, interests, reading level, accessibility needs and the devices they are likely to use. These two decisions shape everything else, from the tone of the writing to the size of the buttons. Next the requirements are listed: the content needed, the functions the site must offer, such as a search box, a form or online payment, and any constraints on cost and time. A site map is then drawn, usually as a tree, showing every page and how the pages link, which makes sure no page is forgotten or unreachable and fixes the navigation structure. Storyboards or wireframes sketch the layout of each page, showing where the menu, headings, text, images and forms will appear, and annotations add details such as fonts, colours, link destinations and alt text. Agreeing a house style at this stage keeps every page consistent, and showing storyboards or a simple prototype to the client and to users finds problems while they are still cheap to fix.

## Navigation and usability

The navigation structure describes how users move between pages. In a linear structure pages are visited in a fixed order using Next and Previous links, which suits processes such as a checkout or a multi-page application form where every step must be completed in turn. In a hierarchical structure the home page links to the main sections, each of which links to its own sub-pages; this is easy to understand and suits most organisational sites, although deep hierarchies need many clicks. A non-linear or mesh structure lets pages link freely to related pages, which suits wikis and news sites but can leave users lost. Most real sites combine these, for example browsing products hierarchically and then paying through a linear checkout. Usability is how easily the target audience can complete their tasks. Usable sites keep the menu in the same place on every page with a route back to the home page, keep key information within a few clicks, use clear headings, short paragraphs and readable fonts, and load quickly because images are compressed. Responsive design uses flexible layouts and CSS media queries so that the same content rearranges itself to fit a phone, tablet or desktop screen without sideways scrolling. Helpful, specific error messages also improve usability.

## Accessibility and WCAG

An accessible website can be used by everyone, including people with visual, hearing, motor or cognitive impairments, and people using assistive technology such as screen readers, magnifiers or switch devices. The Web Content Accessibility Guidelines, published by the World Wide Web Consortium, are the international standard. The current version is WCAG 2.2, and its success criteria are graded at levels A, AA and AAA. They are organised under four principles. Perceivable means content can be seen or heard in some form, through alt text for images, captions and transcripts for audio and video, sufficient colour contrast and text that can be enlarged. Operable means every control can be used, including by keyboard alone, with no content that flashes in a way that could trigger seizures and with enough time to finish tasks. Understandable means plain language, predictable and consistent navigation, labelled form fields and error messages that explain how to fix the problem. Robust means valid, semantic HTML that assistive technologies can interpret reliably. In the UK, public sector bodies such as councils, health trusts and universities are legally required (schools are only partly covered) to meet WCAG level AA and publish an accessibility statement, and disability discrimination law requires other service providers to make reasonable adjustments, so accessibility is a legal as well as an ethical issue.

## Static pages, web applications and client-side processing

A static web page is stored on the server as a finished file and is simply retrieved and displayed; it requires no calculation and no input from the user. A web application, by contrast, is a program accessed through a browser that processes what the user enters, such as an online banking service or a ticket booking system, and it usually keeps its state in a database so that the data changes as people use it. The processing in a web application is divided between the client and the server. Client-side processing is carried out by the browser on the user's own device, using HTML form attributes and JavaScript, without needing any further resources from the server. Its main job is to respond to the user quickly: checking that required fields have been completed, that values have the right type, length, format and range, that related fields have been completed together, and that a time limit has not expired, then displaying a clear message if anything is wrong. It can also change the page as the user works, for example updating a total or revealing extra questions. Only when all checks pass is the data sent, which avoids wasted requests, reduces load on the server and network, and gives the user instant feedback.

## Server-side processing

Server-side processing happens on the web server after the browser has submitted a request. A script, written in a language such as PHP, C# with ASP.NET, Python or JavaScript running on Node.js, receives the data and identifies the transaction required, such as reserving two places on a class. It then fetches any data it needs from the database, for instance the number of places remaining, carries out the calculation, and writes the updated values back along with a record of the booking. If payment is involved, the server contacts a payment provider to authorise the transaction. Should any stage fail, for example a declined card or too few places left, the database is rolled back to its earlier state so that it remains consistent. Finally the server generates a response page, either a confirmation or an explanation of the problem, and sends it back for the browser to display. Server-side processing is essential for anything involving shared or confidential data, because the database and the program code remain on the server where users cannot see or alter them. It is also the only place where some checks can be made, such as whether a username already exists or whether a password matches the stored record.

### Validating form input on both sides

Validation checks that input is sensible and in the expected form; it cannot prove that data is correct, only that it is reasonable. On the client, HTML attributes and scripts provide presence, type, length, range and format checks. In HTML, required acts as a presence check, min and max as a range check, maxlength as a length check and pattern as a format check, while input types such as email and number restrict what can be entered. These checks are fast and convenient, but they are under the user's control: JavaScript can be turned off, the page can be edited using the browser's developer tools, and a request can be sent to the server without using the form at all. For this reason the server must repeat every important check before storing or using the data. Server-side validation protects the database from invalid entries and from deliberately malicious input, such as code inserted into a text field in an attempt at SQL injection. The best practice is therefore to validate twice: on the client for speed and a better user experience, and on the server for security and integrity. Error messages should appear next to the field concerned and say exactly what is wrong and how to correct it.

### Testing, publishing and maintenance

Before a website goes live it is tested systematically against a test plan that lists each test, the data used and the expected result. Functional testing follows every link to find broken links, which lead to missing pages and error messages such as 404 Not Found, and checks that forms accept valid data, reject invalid and extreme data with helpful messages, and submit correctly. Compatibility testing views the site in several browsers and on phones, tablets and desktops, because each may display pages slightly differently. Accessibility testing uses only a keyboard, a screen reader and a contrast checker, and code validation checks the HTML and CSS against the published standards. Performance testing measures loading times, including on a slow mobile connection, and usability testing asks real members of the target audience to complete tasks while observers note difficulties. To publish the site, the owner registers a domain name with a registrar, rents space from a web hosting company whose servers are permanently connected to the internet, uploads the files using FTP, SFTP or the host's control panel, points the domain's DNS records at the host and installs a certificate so the site is served securely over HTTPS. The live site is then tested again and maintained: content is updated, broken links are fixed, software is patched and backups are taken.

## T Key terms from the mark schemes

**Target audience**
The group of people a website is designed for, described by factors such as age, interests, abilities, accessibility needs and the devices they use.

**Storyboard**
A sketch of the layout of each web page, showing the position of navigation, text, images and other elements, often with annotations.
`Credited: Wireframe`

**Linear structure**
A navigation structure in which pages are visited in a fixed sequence using next and previous links.
`Credited: Sequential structure`

**Responsive design**
Web design in which the layout adapts automatically to the size of the screen being used.

**Client-side processing**
Processing carried out by the browser on the user's device, such as validating form input or changing the page, without contacting the server.

**Site map**
A diagram showing all the pages of a website and how they are linked.
`Credited: Structure chart of the site`

**Hierarchical structure**
A navigation structure in which the home page links to main sections, which in turn link to their own sub-pages.
`Credited: Tree structure`

**WCAG**
Web Content Accessibility Guidelines: the international standard for accessible web content, based on the principles perceivable, operable, understandable and robust.

**Web application**
An application that runs in a web browser and is delivered from a server, processing user input and usually storing its data in a database on the server, without being installed on the device.
`Credited: Dynamic website`

**Server-side processing**
Processing carried out on the web server after a request is submitted, such as querying and updating a database and generating the response page.

**Broken link**
A hyperlink whose destination does not exist, producing an error such as 404 Not Found.
Credited: Dead link

**Web hosting**
A service that stores a website's files on servers permanently connected to the internet so the site can be accessed by users.

## C C# example

Why validate on both client and server?

```
Console.Write("Postcode: ");
string pc = (Console.ReadLine() ?? "").Trim().ToUpper();
string[] parts = pc.Split(' ');
bool ok = parts.Length == 2 && pc.StartsWith("BT")
          && parts[0].Length >= 3 && parts[0].Length <= 4
          && parts[1].Length == 3 && char.IsDigit(parts[1][0])
          && char.IsLetter(parts[1][1]) && char.IsLetter(parts[1][2]);
for (int i = 2; ok && i < parts[0].Length; i++)
{
    if (!char.IsDigit(parts[0][i])) ok = false;
}
if (ok) Console.WriteLine("Format accepted - now check the database");
else Console.WriteLine("Enter a postcode such as BT74 6DQ");
```

### Exam focus

Questions on client-side and server-side processing are often set in a scenario such as logging in to online banking or buying tickets, and the marks go to answers that apply both terms to that scenario. Say what the browser does, for example checking that both the username and password boxes are filled before anything is sent, and what the server does, such as retrieving the account details from the database, building the page and returning it to the browser. Saying only that one happens on the computer and the other on the server earns little. When explaining why validation is repeated on the server, state that client checks can be bypassed and that the server protects the data. Planning questions usually ask what a site map or storyboard shows and why it is useful, so give both parts. For navigation structures, choose one and justify it for the context given. Accessibility answers should name a specific feature and say whom it helps and how, for example that alt text is read aloud by a screen reader for a blind user. Test plan answers need specific tests with the data and expected outcome, not vague phrases like checking that it works. In extended questions, cover a range of points, link each to the context and finish with a justified conclusion.

## ? Check yourself

1. State two things a developer should find out about the target audience before designing a website. [2]
2. Describe what a site map shows and explain why one is produced before a website is built. [3]
3. A Belfast arts centre is designing its website. It has sections for shows, workshops and visiting, and a ticket-buying process. Describe a suitable navigation structure for the site, justifying your choice. [4]

**Answers:** 1. Any two from: age range (1) / interests or purpose of their visit (1) / reading level or IT skills (1) / accessibility needs, e.g. visual impairment (1) / the devices they use, e.g. mobile phones (1) / language (1). 2. Shows every page in the website (1) and how the pages are linked / the navigation structure (1) / produced first so no page is missing or unreachable and the navigation can be agreed before coding begins (1). 3. Hierarchical/mixed structure identified (1) / home page links to main sections: shows, workshops, visiting (1) / each section links to its own sub-pages, e.g. individual shows (1) / ticket buying uses a linear sequence (select seats, details, payment, confirmation) because steps must be completed in order (1). Justification must relate to the context.